

CS698B: Fundamentals of Data Engineering (Part I)

2026-27-I, IIT Kanpur – Problem Sheet 4



General Instructions:

1. This problem sheet need not be submitted.
2. The questions are meant for practice and self-assessment.
3. Please use this resource honestly to assess your understanding of concepts.

Q1 (Become the Detective). There are four pieces of code given below, all of which are expected to take as input a list of integers (could be positive, zero or negative), and return only the odd numbers, without changing the order of the numbers. Three of the code fragments will do the job properly and one will not. Find out the buggy code and generate a test case where it will give wrong output.

SNo	Code
A	<pre>def odds(nums): out = [] for x in nums: if x % 2 != 0: out.append(x) return out</pre>
B	<pre>def odds(nums): return list(filter(lambda x: x % 2 == 1, nums))</pre>
C	<pre>def odds(nums): return [x for x in nums if x / 2 * 2 + x % 2 == x]</pre>
D	<pre>def odds(nums): return list(filter(lambda x: x & 1, nums))</pre>

Q2 (Become the Detective). There are four pieces of code given below, all of which are expected to take as input a list of integers (could be positive, zero or negative), and return the number of distinct elements. Three of the code fragments will do the job properly and one will not. Find out the buggy code and generate a test case where it will give wrong output.

SNo	Code
A	<pre>def distinct_count(values): return len(set(values))</pre>
B	<pre>def distinct_count(values): seen = [] for i in range(len(values)): if values[i] not in values[i:]: seen.append(values[i]) return len(seen)</pre>
C	<pre>def distinct_count(values): seen = [] for x in values: if x not in seen: seen.extend([x]) return len(seen)</pre>
D	<pre>def distinct_count(values): return len({ x for x in values })</pre>

CS698B: Fundamentals of Data Engineering (Part I)

2026-27-I, IIT Kanpur – Problem Sheet 4

Q3 (Become the Machine). Write the output of the following code snippet exactly, down to whitespaces.

```
class Counter:
    total = 0
    def __init__( self, name ):
        self.name = name
        Counter.total += 1

    def bump( self ):
        self.total += 1

a = Counter( "aa" )
b = Counter( "bb" )
print( Counter.total, a.total, b.total )
a.bump()
print( Counter.total, a.total, b.total )
print( "total" in a.__dict__, "total" in b.__dict__ )
```

Q4 (Become the Detective). The `Bag` class is meant to support two magic methods, firstly `__len__()` to count the number of distinct items (quantity ignored) and the `__contains__()` method (that gets invoked automatically whenever someone uses the `in` keyword e.g. `item in bag`) if an item is present in the bag with strictly positive value for `quantity`. However, the code has bugs. Design a test case that reveals all bugs. Make minimal changes to the code so that these two operations are performed correctly.

```
class Bag:
    def __init__( self, pairs ):
        self.items = dict( pairs )

    def __len__( self ):
        return sum( self.items.values() )

    def __contains__( self, item ):
        return self.items.get( item, 0 )
```

Q5 (Lamb(da) Curry) Consider the following code fragment and answer the parts below.

```
class MakeMultiplier:
    def __init__( self, n ):
        self.n = n
    def __call__( self, x ):
        return self.n * x

def make_mul_lambda( n ):
    return lambda x: n * x

def make_mul_closure( n ):
    def mul( x ):
        return n * x
    return mul

f1 = MakeMultiplier( 3 )
f2 = make_mul_lambda( 3 )
f3 = make_mul_closure( 3 )
```

1. Will `f1( n )`, `f2( n )`, and `f3( n )` yield the same output for all integers `n`? Note that the `__call__()` dunder makes objects of a class “callable” as functions so calling `f1( n )` is okay.
2. If we now set `f1.n = 10`, will `f1( n )`, `f2( n )`, and `f3( n )` yield the same output for all `n`?
3. Can the functions `f1()`, `f2()`, `f3()` each be used with `list( map( ..., [ 1, 2, 3 ] ) )`?

CS698B: Fundamentals of Data Engineering (Part I)

2026-27-I, IIT Kanpur – Problem Sheet 4

Q6 (Reorder the Code). Deeborah created 3 classes `Project`, `Exam`, and `Course`, to manage CS698B grading, as well as a list of objects of all 3 classes, and wants to print their scores polymorphically.

```
work = [ Exam( 80, 0.7 ),
          Project( 50, 0.3 ),
          Course( [ Exam( 90, 0.6 ), Project( 40, 0.4 ) ] ) ]
for w in work:
    print( w )
```

Deeborah wrote the class definitions in a separate Python file. Deeba wanted to “improve” the code and wrote a C program to remove duplicate lines from the file. Unfortunately, that C program also ended up inserting unnecessary line(s) and shuffled up the lines in the file. Reorder the lines in correct order (you may have to use a line more than once), indent properly, identify unnecessary line(s), print the output.

```
class Course:
class Exam:
class Project:
def __init__( self, marks, weight ):
def __init__( self, parts ):
def __str__( self ):
def get_marks( self ):
def get_weight( self ):
def score( self ):
return f"{self.score()}"
return self.marks * self.weight
return str( self.score() )
return sum( p.score() for p in self.parts )
self.marks = marks
self.parts = parts
self.weight = weight
```

Q7 (MyMutable) Deebae defined the `Point` class to perform geometric calculations. Two `Point` objects are supposed to be equal if both their coordinates are identical. Deebae wants `Point` objects to behave as usual when used inside sets i.e., a set of `Point` objects should not contain any duplicates. However, the class definition by Deebae is not achieving this objective and giving errors. Deebae is confused as the `__eq__()` dunder is invoked when checking equality of two objects using the `==` operator so sets should have worked. Fix Deebae’s code so that it works as intended (you may have to define other dunder(s)). **Note that iPython allows the use of a backslash `\` as a *line-continuation* character.** This allows overlong lines to be broken into aesthetically shorter chunks without changing the semantics of the code.

```
class Point:
    def __init__( self, x, y ):
        self.x = x
        self.y = y

    def __eq__( self, other ):
        return isinstance( other, Point ) \
            and self.x == other.x \
            and self.y == other.y

pts = { Point( 1, 2 ), Point( 1, 2 ) }
print( len( pts ) )
print( Point( 1, 2 ) in pts )
```

CS698B: Fundamentals of Data Engineering (Part I)

2026-27-I, IIT Kanpur – Problem Sheet 4

Q8 (Become the Machine) Consider the following code snippet that resembles the `rxw` flags present inside a typical Unix system such as the Ubuntu systems in RM606. Fill in the table below to indicate the state of the variables `m`, `f.bits` after each iteration and if `f.turn_on()` was invoked in that iteration.

```
class Flag:
    def __init__( self, bits ):
        self.bits = bits

    def has( self, mask ):
        return ( self.bits & mask ) != 0

    def turn_on( self, mask ):
        self.bits = self.bits | mask

# Each flag has a dedicated bit
# Representing flags as power-of-two
# integers is a neat way to ensure this
READ, WRITE, EXEC = 1, 2, 4

# No flags set initially
f = Flag( 0 )

ops = [ READ, WRITE, READ, EXEC, WRITE ]

for i, m in enumerate( ops ):
    if not f.has( m ):
        f.turn_on( m )
```

Iteration i	Value of m	Value of f.bits	f.turn_on() executed?
0			
1			
2			
3			
4			